

H5Tuner / TunIO — 병렬 I/O 오토튜닝 비교 기반

HPC 응용의 병렬 I/O 설정값을 자동으로 찾아 주는 도구를 만들고 있고, 그 비교 기준선을 확보하는 단계입니다.

코드를 고치지 않고 설정값만 바꿔서 h5bench 쓰기 성능을 260 → 575 MB/s (+121%) 로 올린 것을 KISTI 누리온에서 실측했습니다.

무슨 문제인가

슈퍼컴퓨터에서 큰 데이터를 쓸 때 성능은 설정값 몇 개에 크게 좌우됩니다. 파일을 몇 개 디스크에 나눠 쓸지, 한 조각을 얼마나 크게 자를지, 몇 개 프로세스가 대표로 모아서 쓸지 같은 것들입니다.

문제는 좋은 값을 아무도 모른다는 것입니다. 세 가지 이유입니다.

- 경우의 수가 많습니다. 파라미터 12 개, 조합 4 억 4 천만 가지입니다.
- 파라미터끼리 얽혀 있습니다. 하나씩 따로 최적을 찾아 합치면 최적이지 않습니다.
- 정답이 상황마다 다릅니다. 응용·프로세스 수·데이터 모양이 바뀌면 최적값이 바뀝니다.

그래서 자동 탐색 도구가 필요합니다.

이 저장소가 하는 일

기존 연구 두 개를 같은 바닥 위에 다시 구현했습니다.

- H5Tuner (HDF Group) — 유전 알고리즘으로 12 개를 전부 탐색합니다.
- TunIO (IPDPS 2024) — 영향 큰 4 개만 골라 탐색하고, 더 나아질 게 없으면 조기 종료합니다.

왜 논문 숫자를 인용하지 않고 다시 만들었나. 두 논문은 다른 기계에서 다른 응용을 다른 방식으로 했습니다. 그 숫자를 나란히 놓으면 차이가 알고리즘 때문인지 환경 때문인지 구분되지 않습니다. 같은 파라미터 공간·목적함수·측정법·예산으로 맞춰야 비교가 성립합니다.

최종 목표는 이 둘보다 나은 새 프레임워크입니다. 지금은 그 기준선을 만드는 단계입니다.

어떻게 동작하나

C 라이브러리가 응용을 재컴파일하지 않고 설정값을 주입합니다. `LD_PRELOAD` 로 HDF5 함수 호출을 가로칩니다.

```
평소:      응용 —H5Fcreate→ libhdf5.so
shim 적용: 응용 —H5Fcreate→ libautotuner.so → libhdf5.so
           ↑ 여기서 설정값 주입
```

파이썬이 탐색 루프를 돕니다. 설정을 정해 응용을 돌리고, 성능을 재고, 다음 설정을 정합니다. 프레임워크는 `--framework` 플래그로 갈아 끼우고 나머지는 전부 공유합니다.

지금까지 확인된 것

전부 누리온 실측입니다.

- 오토튜닝이 작동합니다. 260 → 575 MB/s (+121%), 후보 41 개, 5.6 분.
- TunIO 의 논문 주장이 재현됐습니다. 거의 같은 성능을 32% 적은 시간에 얻고 스스로 멈춥니다.
- 단, 조건이 붙습니다. 사전 지식(파라미터 영향도 순위)이 없으면 3.3% 밖에 못 올립니다. 측정 노이즈가 3.4% 이므로 사실상 실패입니다.
- 그 사전 지식은 같은 응용에서 뽑은 것입니다. 처음 보는 응용에 통하는지는 아직 모릅니다.

5 분 만에 돌려 보기

MPI 도 HDF5 도 클러스터도 필요 없습니다. 파이썬 3.6 이상이면 됩니다.

```
python3 -m autotuner info

python3 -m autotuner run --framework h5tuner --dry-run --seed 1 \
    --generations 8 --population 10

python3 -m autotuner run --framework tunio --dry-run --seed 1 \
    --generations 8 --population 10
```

`--dry-run` 은 합성 성능 모델로 탐색만 돌립니다. 탐색 로직을 이해하는 데는 충분합니다.

저장소 구조

이 저장소는 HDF Group 의 H5Tuner 를 포크한 것입니다. 그래서 세 종류가 섞여 있습니다. 표의 "출처" 가 그것을 구분합니다.

- 신규 — 이 프로젝트에서 새로 작성했습니다.
- 원본·수정 — H5Tuner 것이지만 우리가 상당히 고쳤습니다.
- 원본 잔재 — H5Tuner 것을 그대로 두었습니다. 지금은 쓰지 않습니다.

핵심 — 여기만 보면 됩니다

경로	무엇인가	출처
<code>autotuner/</code>	파이썬 오토튜너. 탐색 루프, 두 프레임워크, 강화학습, 스케줄러 연동이 전부 여기 있습니다	신규
<code>src/</code>	C shim. <code>LD_PRELOAD</code> 로 HDF5 호출을 가로채 설정값을 주입합니다	원본·수정
<code>traces/</code>	실측 데이터와 그림 스크립트	신규
<code>01~07-*.md</code>	문서. 읽는 순서대로 번호가 붙어 있습니다	신규
<code>scripts/</code>	인수인계 tarball 포장	신규

`src/` 를 "수정" 으로 분류한 이유가 있습니다. 가로채기라는 아이디어는 원본 것이지만, 비동기 API 혹 3 개 추가, 설정 파일 경로를 환경변수로 받도록 변경, `MPI_Info` 와 XML 문서 누수 수정이 우리 작업입니다. 그 전에는 h5bench 처럼 비동기 API 를 쓰는 응용에서 혹이 하나도 발동하지 않았습니다.

참고용 — 읽을 가치는 있습니다

경로	무엇인가	출처
<code>xml/</code>	설정 파일 예시 5 종 (Lustre, GPFS, 청킹, IBM)	원본 잔재
<code>examples/</code>	최소 예제 <code>ph5example.c</code> 와 설정 파일	원본 잔재
<code>test/</code>	파라미터별 C 테스트. 현재 링크가 깨져 있습니다 (<code>06-진행-상황.md</code> §4.2)	원본·수정

`xml/` 은 쓰지 않지만 버리지 마십시오. 우리 오토튜너가 `config.xml` 을 직접 생성하므로 실행에 필요하지는 않은데, shim 이 읽는 XML 형식의 참조 문서 역할을 합니다. 계층별로 어떤 태그가 들어가는지가 여기 있습니다.

`examples/ph5example.c` 는 shim 이 살아 있는지 확인할 때 유용합니다. h5bench 보다 훨씬 작아서 문제를 좁히기 좋습니다.

잔재 — 쓰지 않습니다

경로	무엇인가	왜 남겨 두었나
<code>evo/evolve.py</code>	H5Tuner 원조 유전 알고리즘 탐색기	<code>autotuner/</code> 가 대체했습니다. 파이썬 3 으로 되살려 두긴 했습니다
<code>.misc/</code>	Blue Waters 슈퍼컴퓨터 모듈 스크립트	Blue Waters 는 2021 년에 운영을 끝냈습니다. 순수 잔재입니다
<code>config/</code>	autotools 빌드 보조 파일	<code>autogen.sh</code> 가 참조합니다. 지우면 빌드가 깨집니다
<code>m4/</code>	autoconf 매크로 자리	비어 있습니다(<code>.gitkeep</code> 뿐). 빌드 스캐폴딩입니다
<code>lib/</code>	빌드 산출물이 놓이는 자리	<code>libautotuner.so</code> 가 여기 생깁니다
<code>AUTHORS</code> <code>NEWS</code> <code>ChangeLog</code> <code>README</code>	전부 0 바이트	원본이 만들어만 두고 채우지 않았습니다

`evo/evolve.py` 를 왜 지우지 않았나. 원조 H5Tuner 논문이 쓴 탐색기가 이것이고, 우리 `autotuner/` 의 `h5tuner` 구현이 이것과 같은 동작을 하는지가 재현성의 근거이기 때문입니다. 비교 대상으로 남겨 두었습니다.

`.misc/` 는 지워도 됩니다. 아무것도 참조하지 않습니다.

문서

`01-프로젝트-개요.md` 부터 읽으십시오. 15 분이면 됩니다.

#	문서	누가 읽나
01	프로젝트 개요	전원
02	설계 결정	코드를 고칠 사람
03	원본 코드 분석	C shim 을 건드릴 사람
04	누리온 환경	클러스터에서 뭔가 돌릴 사람 전원
05	뉴론 이식	다른 시스템으로 옮길 사람
06	진행 상황	작업 시작 직전 전원
07	후속 연구 계획	연구 방향을 정할 사람

바쁘면 01 → 06 → 04 만 읽어도 됩니다.

반드시 알아야 할 함정 셋

전부 에러를 내지 않고 조용히 틀린 결과를 냅니다. 실기에서만 드러났습니다.

`shim` 이 로드조차 안 될 수 있습니다. `LD_PRELOAD` 는 의존 라이브러리를 못 찾으면 조용히 무시합니다. 그러면 모든 실험이 튜닝 없이 돌아가는데 겉보기엔 정상입니다.

MPI-IO 힌트는 Intel MPI 에서만 작동합니다. OpenMPI 3.1.0 의 ROMIO 에는 Lustre 드라이버가 없어 영향도 상위 파라미터가 전부 무시됩니다.

출력을 자르지 마십시오. `tail -20` 때문에 결정적 메시지를 놓쳐 하루를 버린 적이 있습니다.

도움이 필요한 지점

사전 지식이 다른 응용에 전이되는지가 가장 큰 미해결 질문입니다. 실험 설계는 `07-후속-연구-계획.md` §8.6 과 `04-누리온-환경.md` §7h 에 있습니다.

그 밖에 사전 지식의 비용 회계, 다른 시스템에서의 재현, 벤치마크 확대가 남아 있습니다. 자세한 것은 `01-프로젝트-개요.md` §9 에 있습니다.

라이선스

원본은 HDF Group 의 H5Tuner 입니다 (COPYING , BSD 계열, 2012-2016). src/ , xml/ , examples/ , test/ 가 그 계보이고, 재배포는 저작권 고지를 유지하면 허용됩니다.

autotuner/ 는 이 프로젝트에서 새로 작성했습니다.

TunIO 는 논문만 참고했습니다. 저자 코드가 공개되어 있지 않아 논문 기술만 보고 재구현했고, 해석이 갈린 지점은 02-설계-결정.md §7 에 22 건으로 정리했습니다.

Rajesh 외, *TunIO: An AI-powered Framework for Optimizing HPC I/O*, IPDPS 2024, pp. 494-504. DOI 10.1109/IPDPS57955.2024.00050